

南開大學
Nankai University

南　开　大　学

计 算 机 学 院

操作系统第二次实验报告

物理内存与页表
分配算法及 QEMU 调试

组长：　李珉熠　2413982

组员：　吴宇石　2411080

组员：　赵亚鑫　2413715

2026 年 10 月 23 日

摘要

我们在最小可执行内核基础上，完成了物理内存探测与页描述符初始化、最佳适应页分配器、伙伴页分配器和简化对象分配器，并通过实际运行检查与课程评分验证了结果。默认物理内存初始化及最佳适应分配评分为 20/20；伙伴与对象分配另有独立测试。在分支任务中，我们构建独立带调试信息的 QEMU，以客体 GDB 观察页表和控制寄存器，以宿主 GDB 跟踪取指及数据读取的地址翻译。实测启动映射采用顶层 1GiB 叶子，页表查询无需走满三级。我们在报告中说明课程框架、实际修改、AI 辅助过程和测试结论。

关键词：物理内存；最佳适应；伙伴系统；对象分配；Sv39；QEMU 调试

目录

一、 实验目的 **1**

二、 实验环境与整体逻辑 **1**

三、 实验内容与实现 **2**

（一） 物理内存初始化 page_init 2

（二） First-Fit 的理解及改进 2

（三） Best-Fit 2

（四） Buddy System 3

（五） 简化 SLUB 对象分配 3

（六） 任务规格与 AI 辅助过程 3

四、 分支任务：实际调试 QEMU 的 MMU 路径 **4**

（一） 调试架构 4

（二） SATP 与大页的客体观察 4

（三） 宿主断点与真实翻译 5

（四） 页表遍历与 TLB 原理 5

五、 测试结果 **5**

六、 总结、理论联系与扩展思考 **6**

（一） 内存管理各层的关系 6

（二） 扩展任务 Challenge：硬件的可用物理内存范围的获取方法（思考题） 6

（三） 当前实现的局限与总结 7

七、 关键代码的调试验证 **7**

实验基本信息与小组分工

实验日期：2026-10-23。我们使用的代码来源为：课程 labcode 的 lab2.zip。我们保留了课程自带 Best-Fit 两个测试函数和评分测试源码，关键测试结果与调试数值记录在本报告中，完整运行日志保留在本地。

成员	模块与报告分工
2413982 李珉熠（组长）	环境、物理内存初始化、构建与评分、汇总报告
2411080 吴宇石	First/Best-Fit、Buddy 与简化 SLUB 算法及测试分析
2413715 赵亚鑫	分页切换、内核 GDB 与 QEMU 源码调试、证据整理

我们按以上模块组织小组分工，并使用 Codex 作为 AI 辅助工具，协助阅读框架、实现算法、定位错误和整理文档。我们以实际编译、评分和调试输出核对实验结果，并分别理解和复现所负责的内容；报告中的实现与测试结论以实际运行记录为依据。

一、 实验目的

1. 从 DTB 获取 RAM 范围，建立 Page 描述符，隔离固件、内核、元数据和 DTB 占用。
2. 理解 First-Fit，完成 Best-Fit；从零实现 Buddy 和简化 SLUB，并验证分配、释放及回收。
3. 理解 Sv39、SATP、叶子 PTE、TLB 与页表遍历；同时调试客体内核与宿主 QEMU。

二、 实验环境与整体逻辑

我们的实验环境为 Windows 下的 VMware Ubuntu 24.04，RISC-V GCC 15.1.0、GNU make、系统 QEMU 8.2.2、OpenSBI v1.3、gdb-multiarch。我们还在个人目录中构建了 Ubuntu 源码包中的 QEMU 8.2.2 调试版，用于宿主源码调试；构建后直接运行该二进制，没有执行 make install。

```
QEMU 预装固件/镜像 → ROM → OpenSBI → entry.S
→ 建立 Sv39 映射、写 satp、sfence.vma → 高地址 kern_init
→ DTB 解析 → 选择 pmm_manager → page_init → 分配器 check
→ SLUB 对象测试 → 无限循环
```

内核链接 VA 起点为 0xffffffffc0200000，加载 PA 为 0x80200000，偏移量为 0xffffffff40000000。VA=PA+offset；PADDR 进行反向转换。在实现和调试中，我们分别区分 Page *（描述符指针）、物理地址 PA、虚拟地址 VA 和物理页号 PPN。

我们的实验目录为 `/home/lmy/os/lab2`，工具链位于 `/home/lmy/riscv-elf-toolchains/bin`。

三、 实验内容与实现

（一） 物理内存初始化 page_init

我们实现了接口 `staticvoidpage_init(void)`。该函数无参数、无返回值，通过全局量 pages、npage 和 va_pa_offset 提供初始化结果。

1. 验证 DTB 的 base、size、加法溢出；当前框架 page2ppn 固定使用 DRAM_BASE，因此明确拒绝其他基址，避免悄悄错误映射。
2. 设置 va_pa_offset，取设备树上界和 PHYSICAL_MEMORY_END 的较小值，再向下按 4096 字节对齐。
3. npage 是管理上界的绝对物理页号，不是 RAM 页数；描述符数量为 npage-nbase。
4. 将描述符数组放在链接符号 end 之后并按页向上对齐，先检查整个数组能否放入受管 RAM；每个描述符先标 reserved、ref=0。
5. 空闲起点放在描述符数组之后，向上对齐。保留 OpenSBI、内核及元数据。当前 DTB 在 RAM 顶部附近，为避免覆盖，保守保留从 DTB 所在页至管理上界的尾段；这是安全但可能浪费部分内存的策略。
6. 仅当存在完整空闲页时调用 init_memmap。框架要求 n>0，因此没有空闲整页时不调用它。

我们在默认 128 MiB RAM 下的实测结果为：管理起点为 `0x80000000`，上界为 `0x88000000`，npage-nbase=32768，sizeof(Page)=40；描述符 PA=0x80207000；完整空闲页区间从 `0x80347000` 到 `0x87e00000`（不含上界），共 31417 页。64 MiB 实测自动缩小管理上界；256 MiB 实测按框架 0x88000000 上界裁剪。评分脚本关于 npage 的预期针对默认内存，不能把默认 20/20 推广为任意内存配置的评分结论。

（二） First-Fit 的理解及改进

我们首先分析了课程提供的 First-Fit：default_init 初始化双向链表和空闲计数；init_memmap 将可用区间组成块，头页 property 保存块长。alloc_pages 从链表头开始选第一块长度 ≥n 的块，分割余块；free_pages 按地址插入，并与前后物理相邻块合并。

我们新增 `PMM=first` 编译选项，便于现场独立运行课程 First-Fit 管理器。实际启用时修正了参考实现中两处尾插后缺少 break 的问题，保留 basic_check 和 default_check 内容不变；First-Fit 及其下层 SLUB 检查已运行通过。

查找时间最坏 O(空闲块数)，地址有序链表便于合并，但不保证最小碎片。可使用分桶、平衡树、位图或最近搜索位置改善搜索成本；也可减少频繁分割。不能保证任何策略都能消除外部碎片。

（三） Best-Fit

我们实现 Best-Fit 时复用框架约定的地址有序空闲链表；遍历所有满足 property≥n 的块，选择 property 最小者。同长时保留首先遇到的低地址块。分配后移除原头、为剩余块设置新头，减少 nr_free；释放时插入并合并相邻空闲块。init_pmm_manager 默认选择 best_fit_pmm_manager。

我们修正了参考链表插入逻辑时发现的尾插入风险：在尾部插入后增加 break，避免循环继续访问刚插入的节点甚至重复插入。除课程原测试外，增加 32 页隔离池测试：分配 32 个单页、耗尽返回 NULL、释放偶数页形成碎片时不能分配 2 页、释放奇数页后重新合并成 32 页，最终

计数恢复。

原 basic_check 与 best_fit_check 内容保持不变；新增 best_fit_verify 包装调用二者之外的附加测试。普通运行和注入评分都真正使用 Best-Fit。改进可采用按长度索引的树/分桶，同时保留按地址合并结构；当前线性查找为 O(k)，仍可能制造小碎片。

（四） Buddy System

我们新增 buddy_pmm.c/h，使用 0 到 15 阶空闲链表，每阶块大小为 2^order 页。初始化把任意长度初始区间拆成物理页号对齐的二次幂块。申请 n 页向上取整到最小二次幂，从足够大的一阶拆分；伙伴页号由 ppn XOR (1«order) 得到。释放仅合并同阶、真正的伙伴，不能仅凭地址相邻就合并。

我们的 alloc_pages(3) 实际占 4 页；释放时使用原始申请长度 3，使 free_pages 按同样的阶数回收。各阶头标记由 PageProperty 与 property 保存；available 统计实际剩余页数。我们通过 `makePMM=buddy` 切换管理器，并在切换前执行 make clean，避免复用另一配置的目标文件。

测试覆盖：零长度和超最大阶请求、3 页向 4 页取整、4 页/8 页对齐、并存块无重叠、释放前后计数恢复；隔离一个 8 页块，拆成 8 个单页至耗尽，逆序释放，重新申请 8 页必须恢复原块。这能检查分裂、伙伴合并和计数，而不仅是“申请返回非空”。

我们还增加了 16 页隔离测试：先申请物理对齐的 16 页块并暂存其余空闲链表，清理测试块管理状态，将第 7 号页设为保留，只把其余 15 页加入测试池。逐页申请 15 次，逐项检查范围、不含保留页、无重复；第 16 次申请应失败。释放 15 页后解除中间页保留并归还，检查合并为 4 阶 16 页块。再申请 3 页和 5 页，分别占 4 页和 8 页，剩余 4 页；释放后恢复 16 页，重新申请整块成功，最后恢复原链表与全局空闲计数。我们通过手动 GDB 断点核对到空闲数 15、0、16、4、16，以及全局计数恢复。

（五） 简化 SLUB 对象分配

我们新增 kmalloc(size_t n)、kfree(void *p)、slub_check()。这是体现 SLUB 主体思想的单 CPU 教学实现，不等同于 Linux 生产级 SLUB。

下层使用当前 pmm_manager 的页分配；上层提供 16、32、64、128、256、512、1024 字节七种缓存。每个 slab 一页，页首为元数据，后面切分为固定长度对象；空闲对象首部存放下一个空闲对象指针。对象 16 字节对齐。超过 1024 字节走连续页大对象路径，申请页数为 ceil((header+n)/4096)，最大接受 1 MiB。大对象仍保留首部元数据，使 kfree 能还原页及长度。

kfree(NULL) 无操作。释放小对象时检查页头、位置、对齐及重复释放，把对象放回空闲链；整 slab 空闲后从缓存链移除并归还底层页。Buddy 模式下底层页请求可能取整，但仍保存原请求页数，以相同请求释放。

测试覆盖各大小类边界 1/16/17/32/33/.../1024/1025、4096 和 9000 字节，逐字节写入和核对防止重叠，逆序释放后空闲页计数恢复；300 个 32 字节对象跨多个 slab，验证数据、同一可用 slab 中的对象复用及所有页回收。我们在早期测试中错误假定任意 slab 释放的对象都是下次首选；核对缓存链选择顺序后，我们改为释放当前优先缓存 slab 中的对象再检验复用。

（六） 任务规格与 AI 辅助过程

我们在持续协作中使用 Codex 辅助阅读、编码、排错和解释，围绕接口、约束与验收标准组织任务。以下提示词是我们对协作需求的整理，便于复现实现和测试思路。

物理页初始化与 Best-Fit。我们要求基于 DTB 初始化 pages、npage 和 va_pa_offset，检查溢出及内存边界，保留固件、内核、描述符和 DTB 占用；Best-Fit 遍历全部合适块选择最小者，保持地址有序及相邻块合并。我们保留课程 basic_check、best_fit_check 与评分测试，另加耗尽、碎片和合并测试。

Buddy 实现提示。“实现独立 pmm_manager：0 到 15 阶，每块为 2^order 页；初始化按绝对物理页号对齐分解区间；申请向上取整并逐阶拆分，释放按 ppn XOR (1«order) 寻找同阶伙伴并递归合并。支持 PMM=buddy，明确原始申请长度的释放约定。”

Buddy 测试提示。“覆盖零与过大请求、3 页申请实际占 4 页、对齐和不重叠；隔离 8 页，拆成 8 个单页至耗尽，逆序释放后重新申请完整 8 页，恢复原状态并核对空闲计数。”

SLUB 实现提示。“在页分配器上实现 kmalloc/kfree，提供 16 到 1024 字节七种缓存，每 slab 一页、对象 16 字节对齐、空闲对象内嵌 next 指针；大对象以带头部的连续页分配，限制 1MiB，完全空闲时归还页，兼容 Best-Fit 和 Buddy。”

SLUB 测试提示。“覆盖大小类边界、零与过大请求、跨页大对象；逐字节检查数据，300 个 32 字节对象跨多个 slab，检查当前可用 slab 内的对象复用，全部释放后核对底层页数，在两种页分配器下分别运行。”

QEMU 调试提示。“在个人目录构建带符号的 QEMU，用宿主 GDB 运行模拟器，客体 GDB 连接 stub；分别观察 kern_init 取指和 boot_dtb 数据读，检查 TLB miss、页表项读取与最终物理地址，排除 is_debug 访问，并对照 Bare 与 Sv39。”

我们通过实际构建与运行核对 AI 辅助生成的代码，发现不符合框架或测试前提的内容后继续修改；测试结果以第五节的实际验证范围为准。

我们在实验中实际修复的问题包括：QEMU 通用 loader 缺少入口交接信息，改用-kernel；入口缺失低地址过渡映射，增加顶层第 2 项恒等映射；工具链明确 rv64gc/lp64d 并改正旧 sfence.vm 别名；课程脚本 CRLF 在 sh 下报错，统一 LF；Best-Fit 尾插入增加 break；纠正 SLUB 复用测试前提；QEMU 源码包缺少可安装 BIOS 资源时使用 disable-install-blobs，运行时-L /usr/share/qemu 复用系统固件。

四、 分支任务：实际调试 QEMU 的 MMU 路径

（一） 调试架构

我们用客体 gdb-multiarch 经 QEMU stub 观察 RISC-V 寄存器和内核，用宿主普通 gdb 运行 x86-64 的 QEMU 进程，并在其 C 源码中设置断点。宿主 GDB 作为 QEMU 父进程，不需要 sudo attach，也避免 ptrace 限制。QEMU 调试版位于 /home/lmy/oslab-tools/qemu-ubuntu-build/qemu-system-riscv64，源码在相邻 qemu-source。我们分别检查了内核启动、页表设置、取指和数据读取的翻译路径。

（二） SATP 与大页的客体观察

我们在客体 GDB 中读到 SATP=0x8000000000080205：MODE=8 表示 Sv39，ASID=0，根表 PPN=0x80205，即物理地址 0x80205000。根表第 511 项位于 0x80205ff8，PTE=0x200000cf；恒等映射第 2 项也为相同 PTE。

PTE 的 PPN=0x80000，对应 PA 基址 0x80000000；低位 0xcf 表示 V/R/W/X/A/D 为 1、U/G 为 0。根表叶子映射 1 GiB。三级页表是最多三级查询，遇到叶子可提前结束。本实验两项

覆盖低地址 0x80000000 起 1 GiB，以及高地址 0xffffffffc0000000 起 1 GiB。真实受管 RAM 只有其中一部分，大页映射范围不等于可分配 RAM 范围。

（三） 宿主断点与真实翻译

我们分别选择 kern_init 的取指和 boot_dtb 的数据读取做两次独立运行。宿主条件断点位于 riscv_cpu_tlb_fill、get_physical_address、PTE 读取后的分支和 tlb_set_page 前。is_debug=0 用于排除调试器自身的内存查询。

我们记录到以下关键输出：

```
WALK_ENTER addr=0x80200000 satp=0 is_debug=0
WALK_ENTER addr=0xffffffffc02000d6 satp=0x8000000000080205 is_debug=0
PTE_READ ... i=0 ptshift=18 base=0x80205000 pte_addr=0x80205ff8 pte=0x200000cf
TRANSLATED VA=0xffffffffc02000d6 PA=0x802000d6 prot=7 cached_size=0x1000
TLB_MISS address=0xffffffffc0206008 access=0 mmu_idx=1
TRANSLATED VA=0xffffffffc0206008 PA=0x80206008 prot=7 cached_size=0x1000
```

取指 access=2，数据读取 access=0，mmu_idx=1 对应此次 S 模式访问。数据读调用栈真实出现 do_ld8_mmu → mmu_lookup → mmu_lookup1 → tlb_fill → riscv_cpu_tlb_fill。

（四） 页表遍历与 TLB 原理

QEMU 8.2 的 target/riscv/cpu_helper.c 中 get_physical_address 先检查权限模式和 SATP 模式；本次 Bare 阶段直接使用原地址。Sv39 时以根表物理地址为 base，计算当前层 index 和 pte_addr=base+index*8，使用 address_space_ldq 读取 8 字节 PTE，再检查合法性。V=0 失败；R/W/X 有叶子权限则转到 leaf；否则提取 PPN，将 base 更新为下级表物理地址，继续循环。因此循环的职责是层级遍历，不是重复猜地址。框架顶层即为叶子，所以这次循环 i=0 时结束。

此处 ptshift=18 是还要保留的下级 VPN 位数，两层各 9 位，共 18 位；不是虚拟地址总偏移。1 GiB 叶子的物理地址由 PPN 高位、虚拟地址的低级 VPN 和 12 位字节偏移组成，等价于 PA=0x80000000+(VA & 0x3fffffff)。对数据读得到 0x80206008。

软件 TLB 查找位于 accel/tcg/cputlb.c。mmu_lookup1 读取并比较 TLB 标签，不命中时先尝试 victim_tlb_hit，再通过 tlb_fill 调用架构的 riscv_cpu_tlb_fill；成功后 tlb_set_page 缓存结果。取指另由 get_page_addr_code_hostp/probe_access_internal 路径进入。get_physical_address 不是每次访问必经函数：命中缓存时可以绕过页表遍历，TCG 翻译块缓存也会影响取指路径。

QEMU 软件 TLB 承担客体地址转换及访问权限缓存，并保存加速宿主访问需要的信息；它不等于实体 RISC-V 处理器某一型号的硬件 TLB，也不等于宿主 x86 CPU 自己的 TLB。本次大页翻译虽然使用 1 GiB 叶子，软件缓存填充实测 cached_size 为 0x1000，不能把二者混为相同页大小。进入内核前 Bare 模式虽然不做 Sv39 遍历，QEMU 仍可能调用软件 TLB 填充，这正是日志中 SATP=0 时也有 TLB_MISS 的原因。

五、 测试结果

测试	实测结果	验证内容
课程 page_init 与 Best-Fit 评分	20/20	默认 128MiB 配置下的课程评分
Best-Fit 原测试与附加 32 页碎片/合并	PASS	耗尽、连续性、释放后合并和计数恢复
First-Fit 课程自检与 SLUB	PASS	独立构建运行、页分配与对象回收
Buddy 取整/对齐/耗尽/合并	PASS	8 页拆分；16 页保留页、唯一性、3/5 页取整与全量恢复
SLUB 尺寸边界/数据/多 slab/复用/回收	Best-Fit、Buddy 两种下层均 PASS	数据不重叠、对象复用、底层页归还
64MiB、256MiB 启动与内存边界	启动及测试成功	64MiB 自动缩小，256MiB 按 128MiB 上界裁剪
客体 GDB 与 SATP/PTE	高地址断点及根表值已确认	SATP=0x8000000000080205，PTE=0x200000cf
宿主 QEMU 取指与数据读翻译	条件断点命中并确认 PA	分别得到 PA=0x802000d6 与 0x80206008

我们在限时验证中使用 timeout 结束运行，在宿主跟踪完成后由调试脚本主动结束 QEMU，因此相应的 timeout 和 killed 信息属于验证流程结束。我们在本地保留了真实文本日志；测试范围为上表所列场景，尚未覆盖生产级并发、任意恶意 DTB 和全部硬件配置。

六、 总结、理论联系与扩展思考

（一） 内存管理各层的关系

页级分配器解决“哪些连续物理页可以使用”，SLUB 解决“如何把页切成对象”，页表解决“CPU 地址如何翻译”，TLB 解决“如何缓存翻译”；这些是不同层次。reserved 防止把固件/内核/描述符作为空闲内存发出；property 只在空闲块头解释，不能拿任意页的值当块长。

（二） 扩展任务 Challenge：硬件的可用物理内存范围的获取方法（思考题）

问题：如果 OS 无法提前知道当前硬件的可用物理内存范围，有何办法让 OS 获取可用物理内存范围？

我们的思路是由内核在启动时读取固件或启动程序提供的硬件描述，而不是把某一种机器的内存容量写死。可用的信息来源包括设备树 DTB、UEFI 内存映射或平台专用固件协议。DTB 是设备树的二进制数据，用于描述硬件；UEFI 是一种固件接口，其内存映射可描述各内存区域的类型和用途。

具体处理分为三步：

1. 获取内存布局：启动程序传入硬件描述的位置，内核解析其中的 RAM 起始地址、大小以及可获得的区域属性，检查边界和地址计算是否溢出。
2. 确定可分配范围：从 RAM 中排除固件、内核、启动栈、页表、页描述符数组、设备树以及

其他保留区域；设备的地址窗口也不能作为普通空闲 RAM 分配。因此，硬件提供的 RAM 总范围不等于当前可分配的空闲范围。

3. 建立管理结构：将剩余区间的起点向上、终点向下按页对齐，为完整物理页建立管理状态，再交给页分配器。若硬件存在多段不连续 RAM，应分别记录，不能把中间的地址空洞当作内存。

在我们本次实验中，启动环境通过 a1 寄存器传入 DTB 的物理地址，内核解析 memory 节点的 reg 属性，取得 RAM 起点 0x80000000 和大小 128 MiB。我们再结合内核布局计算空闲范围，最终将 [0x80347000,0x87e00000) 的完整页交给分配器。

如果没有标准硬件描述，我们可以考虑使用平台专用的内存控制器查询接口，或在已知平台规则和异常处理机制下进行受控探测。但不能盲目向任意物理地址读写测试，否则可能覆盖已有数据、访问设备寄存器或触发访问异常。

我们当前的 DTB 解析只识别一个 memory 节点中的一段内存范围，尚未完整处理多段 RAM、设备树保留表和 reserved-memory 节点。后续可扩展这些信息的解析，建立更完整的可用与保留区间列表。

（三） 当前实现的局限与总结

我们本次尚未实现进程地址空间隔离、按需分页、页面置换、系统调用、调度、文件系统，以及多核/中断环境下的分配器同步。过渡恒等映射和宽权限 1GiB 启动页适合教学启动，不是最终内核应长期采用的最小权限布局。

通过本次实验，我们完成了可管理物理页的建立、Best-Fit 接入、Buddy 和对象分配，并在 QEMU 源码中观察了真实地址转换的关键路径。

参考：课程练习、分支任务、QEMU v8.2.2 MMU 源码、QEMU 软件 TLB。我们实际调试时采用 Ubuntu 重打包的 8.2.2 源码。

七、 关键代码的调试验证

我们进一步通过实际`makegdb` 入口检查启动状态，并用断点观察分配算法的区别。`makegdb` 内部调用工具链 GDB 加载符号、设置 RISC-V 架构并连接 1234 端口；我们读到的 SATP 和 PTE 与前述数值一致。

我们在 best_fit_check 第 237 行申请 1 页之前，通过 GDB 确认两个空闲块相对 p0 偏移分别为 1 和 4，长度分别为 2 和 1，总空闲 3 页；执行该行后 p1-p0=4、剩余 2 页；接着第 238 行申请连续 2 页仍成功，空闲页数归零。因此第 239 行 assert(p0 + 4 == p1) 结合前一行连续申请，明确验证最小合适块选择，与 First-Fit 的首次合适块选择不同。我们还通过 GDB 确认描述符大小为 40 字节，page_link 偏移为 24 字节。

Page 管理标记不是 PTE 硬件权限；单张 Sv39 表为 4KiB、512 项，而完整多级结构按实际映射分配；启动根表使用 1GiB 叶子所以不走满三级。测试断言属于覆盖特定行为的证据，不能代替任意非法释放或并发正确性的证明。这些观察帮助我们把分配策略、链表结构和实际执行结果对应起来。